

RISC-V

Chương 1. Tổng quan về kiến trúc RISC-V

1.1. Giới thiệu về RISC-V

RISC-V là một kiến trúc tập lệnh, hay **Instruction Set Architecture (ISA)**, được xây dựng theo triết lý **Reduced Instruction Set Computer (RISC)**. ISA đóng vai trò là lớp giao tiếp giữa phần mềm và phần cứng, quy định các loại lệnh mà bộ xử lý hỗ trợ, cách mã hóa lệnh, hệ thống thanh ghi, cách truy cập bộ nhớ và cách thay đổi luồng thực thi chương trình.

Khác với nhiều kiến trúc thương mại có bản quyền, RISC-V được xây dựng theo hướng mở và có khả năng mở rộng. Kiến trúc cơ sở có thể được bổ sung thêm các phần mở rộng để hỗ trợ những chức năng khác nhau như phép toán nhân chia, số thực, xử lý vector hoặc các cơ chế đặc biệt khác.

Trong thiết kế này, bộ xử lý sử dụng tập lệnh cơ sở **RV32I**. Trong đó:

- **RV** biểu thị kiến trúc RISC-V.
- **32** biểu thị chiều rộng thanh ghi và đường dữ liệu cơ bản là 32 bit.
- **I** biểu thị tập lệnh số nguyên cơ sở.

Do đó, các thanh ghi tổng quát, ALU, Program Counter và phần lớn đường dữ liệu của bộ xử lý đều có độ rộng 32 bit.

1.2. Đặc điểm của kiến trúc RISC

Kiến trúc RISC hướng đến việc sử dụng các lệnh đơn giản, có cấu trúc tương đối đồng nhất và thuận lợi cho việc triển khai phần cứng.

Một số đặc điểm chính bao gồm:

- Các lệnh có độ dài cố định.
- Số lượng định dạng lệnh tương đối ít.
- Các phép toán số học và logic chủ yếu thực hiện trên các thanh ghi.
- Bộ nhớ thường được truy cập thông qua các lệnh load và store.
- Việc giải mã lệnh tương đối đơn giản.
- Thuận lợi cho việc triển khai bộ xử lý theo kiến trúc pipeline.

Đối với RV32I, mỗi instruction có độ dài: 32 bit=4 byte.

Do đó, trong trường hợp chương trình thực hiện tuần tự, Program Counter thường tăng: $PC_{next}=PC+4$.

ví dụ:

```
0x00008000
0x00008004
0x00008008
0x0000800C
...
```

1.3. Mô hình Load/Store của RISC-V

RISC-V sử dụng kiến trúc **load/store**. Điều này có nghĩa là các phép toán số học và logic không thao tác trực tiếp trên dữ liệu trong bộ nhớ.

Ví dụ, muốn cộng hai giá trị, trước hết các giá trị cần được đưa vào thanh ghi, sau đó ALU thực hiện phép cộng.

Lệnh:

```
add x3, x1, x2
```

có ý nghĩa: $x3 = x1 + x2$.

Trong khi việc truy cập bộ nhớ được thực hiện bằng các lệnh riêng như:

```
lw x5, 0(x1)
```

và:

```
sw x5, 0(x1)
```

Trong đó:

- **Lw** đọc dữ liệu từ bộ nhớ vào thanh ghi. Trong ví dụ thực hiện lưu giá trị của địa chỉ thanh ghi được lưu trong x1 và thanh ghi x5.
- **Sw** ghi dữ liệu từ thanh ghi xuống bộ nhớ. Trong ví dụ thực hiện lưu giá trị của thanh ghi x5 vào thanh ghi có địa chỉ được lưu trong thanh ghi x1.

Mô hình này giúp datapath rõ ràng hơn vì ALU chủ yếu thực hiện tính toán trên dữ liệu lấy từ Register File.

1.4. Hệ thống thanh ghi của RV32I

RV32I có **32 thanh ghi tổng quát**, được đánh số từ:

```
x0 → x31
```

Mỗi thanh ghi có độ rộng 32 bit.

Có thể biểu diễn: 32 registers × 32 bit.

Thanh ghi **x0** là một trường hợp đặc biệt: $x0 = 0$

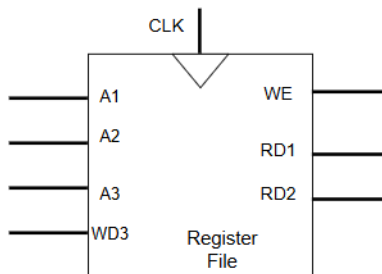
Giá trị của **x0** luôn bằng 0. Mọi thao tác ghi vào **x0** đều bị bỏ qua.

Ví dụ:

```
addi x1, x0, 5
```

Thực hiện: $x1 = x0 + 5$. Do $x0 = 0 \Rightarrow x1 = 5$.

Trong phần cứng, Register File thường có hai cổng đọc và một cổng ghi:



```
A1 → địa chỉ rs1
A2 → địa chỉ rs2
A3 → địa chỉ rd

RD1 → dữ liệu đọc từ rs1
RD2 → dữ liệu đọc từ rs2
WD3 → dữ liệu ghi vào rd
```

Các trường địa chỉ thanh ghi có độ rộng 5 bit vì: $2^5=32$ đủ để biểu diễn 32 thanh ghi.

1.5. Program Counter

Program Counter (PC) là thanh ghi giữ địa chỉ liên quan đến luồng thực thi instruction.

Trong quá trình thực thi tuần tự: $PC_{next}=PC+4$.

Ví dụ:

```
PC = 0x00000000
```

instruction tiếp theo nằm tại:

```
0x00000004
```

Tuy nhiên, với các lệnh thay đổi luồng chương trình như:

```
beq
jal
```

PC có thể nhận một địa chỉ khác thay vì $PC + 4$.

Ví dụ với lệnh nhánh:

```
beq x1, x2, offset
```

nếu điều kiện đúng: $PC_{next}=PC+offset$.

Ngược lại: $PC_{next}=PC+4$.

Do đó, PC là thành phần quan trọng trong việc xác định instruction nào được lấy từ Instruction Memory.

1.6. Cấu trúc instruction trong RV32I

Mỗi instruction RV32I có độ dài 32 bit. Tùy loại lệnh, 32 bit này được chia thành các trường khác nhau.

Các trường phổ biến bao gồm:

- **opcode**: xác định nhóm instruction.
- **rd**: thanh ghi đích.
- **rs1**: thanh ghi nguồn thứ nhất.
- **rs2**: thanh ghi nguồn thứ hai.
- **funct3**: cung cấp thêm thông tin để xác định phép toán.
- **funct7**: phân biệt thêm các instruction có cùng **opcode** và **funct3**.
- **immediate**: hằng số hoặc độ lệch được mã hóa trực tiếp trong instruction.

Các định dạng chính được sử dụng trong thiết kế gồm:

R-type
I-type
S-type
B-type
J-type

1.7. Định dạng R-type

R-type chủ yếu được sử dụng cho các phép toán giữa hai thanh ghi.

Cấu trúc:

31 : 25	24 : 20	19 : 15	14 : 12	11 : 7	6 : 0
funct7	rs2	rs1	funct3	rd	opcode

Các trường:

funct7 : 7 bit
rs2 : 5 bit
rs1 : 5 bit
funct3 : 3 bit
rd : 5 bit
opcode : 7 bit

Ví dụ:

```
add x3, x1, x2
```

Thực hiện: $x3 = x1 + x2$.

Trong instruction này:

rs1 = x1
rs2 = x2
rd = x3

Một số instruction R-type được sử dụng trong thiết kế gồm:

```
add  
sub  
and  
or  
slt
```

1.8. Định dạng I-type

I-type được sử dụng cho một số lệnh số học với immediate và các lệnh load.

Cấu trúc:

31 : 20	19 : 15	14 : 12	11 : 7	6 : 0
Immediate	rs1	funct3	rd	opcode

Ví dụ:

```
addi x1, x0, 5
```

Thực hiện: $x1 = x0 + 5$.

ALU nhận:

```
SrcA = giá trị x0  
SrcB = immediate = 5
```

Một ví dụ khác:

```
lw x9, 0(x8)
```

ALU trước tiên tính địa chỉ: $\text{Address} = x8 + 0$. Sau đó Data Memory được đọc và dữ liệu được ghi vào `x9`.

1.9. Định dạng S-type

S-type được dùng cho các lệnh store, chẳng hạn:

```
sw x3, 0(x8)
```

Cấu trúc:

31 : 25	24 : 20	19 : 15	14 : 12	11 : 7	6 : 0
imm[11:5]	rs2	rs1	funct3	imm[0:4]	opcode

Immediate bị chia thành hai phần trong instruction và phải được ghép lại trong khối Extend.

Lệnh:

```
sw x3, 0(x8)
```

Có ý nghĩa: $\text{Memory}[x8+0] = x3$.

Trong đó ALU được dùng để tính địa chỉ bộ nhớ: $\text{Address} = \text{rs1} + \text{ImmExt}$.

1.10. Định dạng B-type

B-type được sử dụng cho các instruction branch có điều kiện.

Cấu trúc:

31 : 25	24 : 20	19 : 15	14 : 12	11 : 7	6 : 0
imm{12,[10:5]}	rs2	rs1	funct3	imm{[4:1],11}	opcode

Ví dụ:

```
beq x1, x2, offset
```

Lệnh này kiểm tra: $x1=x2$

Nếu hai giá trị bằng nhau, chương trình nhảy đến: $PC_{target}=PC+ImmExt$

Nếu không: $PC_{next}=PC+4$

Immediate trong B-type không nằm liên tục trong instruction mà được phân bố ở nhiều vị trí khác nhau. Khối Extend phải ghép các bit đúng thứ tự và sign-extend lên 32 bit.

Trong thiết kế, `beq` thường sử dụng ALU để thực hiện: $rs1-rs2$

Nếu kết quả bằng 0 thì:

```
Zero = 1
```

Và điều kiện branch được thỏa mãn.

1.11. Định dạng J-type

J-type được sử dụng cho instruction nhảy như:

```
jal rd, offset
```

Lệnh `jal` thực hiện hai thao tác: $rd=PC+4$.

và: $PC=PC+offset$.

Ví dụ:

```
jal x11, +8
```

Nếu instruction nằm tại:

```
0x00008038
```

thì:

```
x11 = 0x0000803C
PC  = 0x00008040
```

Giá trị ghi vào `rd` được sử dụng như địa chỉ quay về khi `jal` được dùng để gọi hàm.

1.12. Immediate và khối Extend

Các instruction như `addi`, `lw`, `sw`, `beq` và `jal` đều chứa immediate.

Tuy nhiên vị trí các bit immediate khác nhau tùy định dạng instruction. Vì vậy cần một khối **Extend** để:

1. Trích xuất các bit immediate.
2. Ghép lại đúng thứ tự.
3. Mở rộng dấu lên 32 bit.

Trong thiết kế có thể sử dụng tín hiệu:

```
ImmSrc[1:0]
```

để chọn định dạng immediate:

ImmSrc	Loại	Ví dụ
00	I-type	addi, lw
01	S-type	sw
10	B-type	beq
11	J-type	jal

Ví dụ đối với I-type:

```
{{20{Instruction[31]}}, Instruction[31:20]}
```

Bit `Instruction[31]` được lặp lại 20 lần để thực hiện sign extension.

1.13. Các nhóm instruction được sử dụng trong thiết kế

Trong phạm vi bộ xử lý được xây dựng, các instruction chính có thể được chia thành các nhóm sau.

Nhóm số học và logic

```
add
sub
and
or
slt
addi
```

Các instruction này chủ yếu sử dụng ALU để tạo kết quả ghi về Register File.

Ví dụ:

```
add x3, x1, x2
```

Nhóm truy cập bộ nhớ

```
lw
sw
```

Ví dụ:

```
lw x9, 0(x8)
```

Thực hiện: $\text{Address} = x8 + 0$. Sau đó: $x9 = \text{Memory}[\text{Address}]$

Còn:

```
sw x3, 0(x8)
```

Thực hiện: $\text{Memory}[x8 + 0] = x3$.

Nhóm điều khiển luồng chương trình

```
beq
jal
```

`beq` thay đổi PC nếu điều kiện bằng nhau được thỏa mãn.

`jal` thực hiện nhảy vô điều kiện đồng thời lưu địa chỉ quay về.

1.14. Vai trò của ALU

ALU, hay **Arithmetic Logic Unit**, là khối thực hiện các phép toán số học và logic.

Trong thiết kế, ALU hỗ trợ các phép:

```
ADD
SUB
OR
AND
SLT
```

Một cách mã hóa `ALUControl` 4 bit có thể là:

<code>ALUControl</code>	Phép toán
<code>0000</code>	ADD
<code>0001</code>	SUB
<code>0010</code>	OR
<code>0011</code>	AND
<code>0101</code>	SLT

ALU không chỉ được sử dụng cho các instruction R-type.

Ví dụ:

```
addi x1, x0, 5
```

ALU thực hiện: `x1 = 0+5`.

Đối với:

```
lw x9, 0(x8)
```

ALU thực hiện: `x8+0` để tính địa chỉ bộ nhớ.

Đối với:

```
beq x1, x2, offset
```

ALU có thể thực hiện: `x1-x2` để xác định hai thanh ghi có bằng nhau hay không.

Như vậy ALU được tái sử dụng cho nhiều mục đích khác nhau trong datapath.

1.15. Các cờ trạng thái của ALU

Ngoài kết quả chính, ALU có thể tạo ra một số cờ trạng thái như:

```
Zero
Carry
```

```
Negative
Overflow
```

Zero

`Zero = 1` khi kết quả ALU bằng 0.

Ví dụ: $5-5=0$

Thì:

```
Zero = 1
```

Cờ này có thể được dùng để xử lý `beq`.

Negative

Cho biết bit dấu của kết quả:

```
Negative = ALUResult[31]
```

đối với số signed 32 bit.

Carry

Cho biết carry-out của phép toán unsigned.

Overflow

Cho biết kết quả phép toán signed vượt khỏi miền biểu diễn 32 bit có dấu.

Các cờ này hỗ trợ việc kiểm tra trạng thái của phép toán và có thể được sử dụng trong các instruction điều kiện hoặc phục vụ quá trình kiểm thử.

1.16. Control Unit

Control Unit có nhiệm vụ giải mã instruction và tạo ra các tín hiệu điều khiển cho datapath.

Trong thiết kế, Control Unit có thể được chia thành hai phần:

```
Main Decoder
ALU Decoder
```

Main Decoder sử dụng:

```
opcode
```

để xác định nhóm instruction và tạo các tín hiệu như:

```
RegWrite
ResultSrc
MemWrite
Jump
Branch
ALUSrc
ImmSrc
ALUOp
```

ALU Decoder tiếp tục sử dụng:

```
ALUOp  
funct3  
funct7  
opcode
```

để tạo:

```
ALUControl
```

Ví dụ với:

```
sub x4, x3, x2
```

Main Decoder nhận biết đây là R-type:

```
opcode = 0110011
```

sau đó ALU Decoder sử dụng `funct3` và `funct7` để xác định phép toán cuối cùng là:

```
SUB
```

1.17. Chu trình thực thi instruction ở mức tổng quát

Mặc dù Single-Cycle, Multicycle và Pipeline tổ chức thời gian thực thi khác nhau, về mặt chức năng một instruction thường trải qua các công việc sau:

```
Instruction Fetch  
↓  
Instruction Decode  
↓  
Read Register  
↓  
Execute  
↓  
Memory Access  
↓  
Write Back
```

Có thể gọi tắt thành:

```
IF → ID → EX → MEM → WB
```

Trong đó:

- **IF – Instruction Fetch:** lấy instruction từ bộ nhớ.
- **ID – Instruction Decode:** giải mã instruction và đọc Register File.
- **EX – Execute:** ALU thực hiện phép toán hoặc tính địa chỉ.
- **MEM – Memory Access:** truy cập Data Memory khi cần.
- **WB – Write Back:** ghi kết quả về Register File.

Không phải instruction nào cũng cần sử dụng đầy đủ tất cả các công đoạn.

Ví dụ `add` :

IF → ID → EX → WB

Trong khi **lw** cần:

IF → ID → EX → MEM → WB

và **sw** không cần Write Back:

IF → ID → EX → MEM

Các kiến trúc Single-Cycle, Multicycle và Pipeline khác nhau chủ yếu ở **cách tổ chức các công đoạn này theo thời gian**.

1.18. Cơ sở để phát triển các kiến trúc bộ xử lý

Từ cùng một tập lệnh RV32I và cùng các thành phần cơ bản như:

PC
Instruction Memory
Register File
Extend
ALU
Data Memory
Control Unit

Có thể xây dựng các vi kiến trúc khác nhau.

Trong **Single-Cycle Processor**, toàn bộ các công đoạn cần thiết của một instruction được hoàn thành trong một chu kỳ clock.

Trong **Multicycle Processor**, quá trình thực hiện một instruction được chia thành nhiều chu kỳ nhỏ hơn và phần cứng có thể được tái sử dụng giữa các chu kỳ.

Trong **Pipeline Processor**, quá trình xử lý được chia thành nhiều stage và nhiều instruction có thể đồng thời tồn tại trong bộ xử lý ở các stage khác nhau.

Ba kiến trúc này vẫn thực hiện cùng một ISA. Sự khác biệt nằm ở cách tổ chức datapath, control và thời gian thực thi instruction.

Trong các chương tiếp theo, ba kiến trúc trên sẽ lần lượt được phân tích và triển khai, bắt đầu từ kiến trúc **Single-Cycle**, sau đó phát triển thành **Multicycle** và cuối cùng là **Pipeline RISC-V**.

Chương 2. Bộ xử lý RISC-V Single-Cycle

2.1. Khái niệm bộ xử lý Single-Cycle

Bộ xử lý **Single-Cycle** là kiến trúc trong đó mỗi instruction được hoàn thành trong đúng **một chu kỳ clock**. Trong một chu kỳ duy nhất, bộ xử lý phải thực hiện toàn bộ các công đoạn cần thiết như lấy lệnh, giải mã, đọc thanh ghi, thực hiện phép toán, truy cập bộ nhớ nếu cần và ghi kết quả về Register File.

Về mặt chức năng, một instruction có thể trải qua các bước:

Instruction Fetch
↓
Instruction Decode
↓
Read Register
↓
Execute
↓
Memory Access

Write Back

Trong kiến trúc Single-Cycle, tất cả các bước trên được hoàn thành trước cạnh clock tiếp theo.

Ví dụ đối với lệnh:

```
lw x5, 0(x1)
```

trong cùng một chu kỳ clock, bộ xử lý phải:

1. Đọc instruction từ Instruction Memory
2. Giải mã opcode
3. Đọc x1 từ Register File
4. Mở rộng immediate
5. ALU tính địa chỉ x1 + immediate
6. Đọc Data Memory tại địa chỉ đó
7. Ghi dữ liệu đọc được vào x5

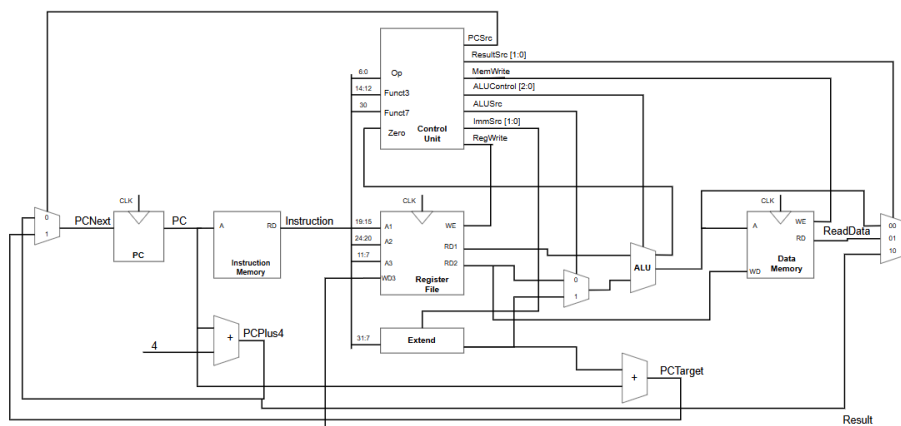
Do đó thời gian của một chu kỳ clock phải đủ dài để instruction có đường truyền dữ liệu dài nhất hoàn thành.

2.2. Kiến trúc tổng thể của Single-Cycle Processor

Một datapath Single-Cycle RV32I cơ bản gồm các khối chính:

Program Counter
Instruction Memory
Register File
Immediate Extend
ALU
Data Memory
Control Unit
Multiplexer
Adder

Lưuồng dữ liệu tổng quát có thể biểu diễn như sau:



Mỗi khối đảm nhiệm một chức năng riêng, nhưng toàn bộ datapath hoạt động tổng hợp trong cùng một chu kỳ.

2.3. Program Counter và Instruction Memory

Program Counter giữ địa chỉ instruction hiện tại. Trong trường hợp thực thi tuần tự: $PC_{next} = PC + 4$. Do mỗi instruction RV32I dài 32 bit, tương đương 4 byte.

Trong thiết kế, chương trình bắt đầu tại:

```
0x0000_8000
```

Instruction Memory chuyển địa chỉ byte thành chỉ số word:

```
word_index= (Address-BASE_ADDRESS) >>2;
```

Ví dụ:

```
0x8000 → memory[0]
0x8004 → memory[1]
0x8008 → memory[2]
```

2.4. Register File và Immediate Extend

Register File gồm 32 thanh ghi 32 bit:

```
x0 → x31
```

trong đó `x0` luôn bằng 0.

Các trường thanh ghi trong instruction gồm:

```
rs1 = Instruction[19:15]
rs2 = Instruction[24:20]
rd  = Instruction[11:7]
```

Đối với các lệnh sử dụng immediate như `addi`, `lw`, `sw`, `beq`, `jal`, khối Extend có nhiệm vụ tạo giá trị immediate 32 bit.

Quy ước:

```
ImmSrc = 00 → I-type
ImmSrc = 01 → S-type
ImmSrc = 10 → B-type
ImmSrc = 11 → J-type
```

2.5. ALU

ALU thực hiện các phép toán số học và logic.

Trong thiết kế, `ALUControl` rộng 4 bit:

```
0000 → ADD
0001 → SUB
0010 → OR
0011 → AND
0101 → SLT
```

Toán hạng thứ hai của ALU được lựa chọn bởi `ALUSrc`:

```
ALUSrc = 0 → RD2
ALUSrc = 1 → ImmExt
```

Ví dụ:

```
add x3, x1, x2
```

sử dụng:

```
SrcA = x1  
SrcB = x2
```

Trong khi:

```
addi x3, x1, 5
```

sử dụng:

```
SrcA = x1  
SrcB = 5
```

2.6. Data Memory và Write Back

Data Memory được sử dụng bởi **lw** và **sw**.

Với **lw**:

```
Address = rs1 + immediate  
ReadData = Memory[Address]  
rd ← ReadData
```

Với **sw**:

```
Address = rs1 + immediate  
Memory[Address] ← rs2
```

MUX **ResultSrc** quyết định dữ liệu ghi về Register File:

```
00 → ALUResult  
01 → ReadData  
10 → PC + 4
```

Do đó:

```
R-type, addi → ALUResult  
lw          → ReadData  
jal        → PC + 4
```

2.7. Control Unit

Control Unit giải mã instruction và tạo các tín hiệu điều khiển datapath.

op	Instruct	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch
3	lw	1	00	1	0	1	0
35	sw	0	01	1	1	X	0
51	R-type	1	XX	0	0	0	0
99	beq	0	10	0	0	X	1

19	addi	1	00	1	0	0	0
----	------	---	----	---	---	---	---

Main Decoder sử dụng:

```
opcode
```

để tạo:

```
RegWrite
ResultSrc
MemWrite
Branch
Jump
ALUSrc
ImmSrc
ALUOp
```

ALU Decoder sử dụng:

```
ALUOp
funct3
funct7
opcode
```

để tạo `ALUControl`.

Đối với branch và jump:

```
PCSrc=Jump | (Branch&Zero);
```

Trong đó `Zero` được tạo bởi ALU khi hai toán hạng của `beq` bằng nhau.

2.8. Hoạt động của các nhóm lệnh

Lệnh R-type:

```
add x3, x1, x2
```

Thực hiện:

```
Register File → ALU → Register File
```

Lệnh `addi`:

```
addi x1, x0, 5
```

Thực hiện:

```
x1 = x0 + 5 = 5
```

Lệnh `lw`:

```
lw x9, 0(x8)
```

Thực hiện:

```
Address = x8 + 0  
x9 = Memory[Address]
```

Lệnh **sw**:

```
sw x3, 0(x8)
```

Thực hiện:

```
Memory[x8] = x3
```

Lệnh **beq**:

```
beq x1, x2, offset
```

Nếu **x1 = x2** thì:

```
PC ← PC + offset
```

Lệnh **jal**:

```
jal rd, offset
```

Thực hiện:

```
rd ← PC + 4  
PC ← PC + offset
```

2.9. Ưu và nhược điểm

Ưu điểm của Single-Cycle là cấu trúc đơn giản, dễ thiết kế và mỗi instruction có: CPI=1.

Tuy nhiên, chu kỳ clock phải đủ dài cho instruction chậm nhất. Ví dụ **lw** phải đi qua:

```
Instruction Memory  
→ Register File  
→ ALU  
→ Data Memory  
→ Write Back
```

Do đó các instruction đơn giản như **add** cũng phải sử dụng cùng một chu kỳ clock dài.

Ngoài ra, Instruction Memory và Data Memory thường phải tách riêng vì trong cùng một cycle bộ xử lý có thể cần truy cập cả hai.

Chương 3. Bộ xử lý RISC-V Multicycle

3.1. Khái niệm

Bộ xử lý **Multicycle** là kiến trúc trong đó một instruction không còn hoàn thành trong một chu kỳ clock duy nhất mà được chia thành nhiều chu kỳ nhỏ.

Ví dụ:

Fetch → Decode → Execute → Memory → Write Back

Tùy loại instruction, số chu kỳ cần thiết sẽ khác nhau.

Ví dụ:

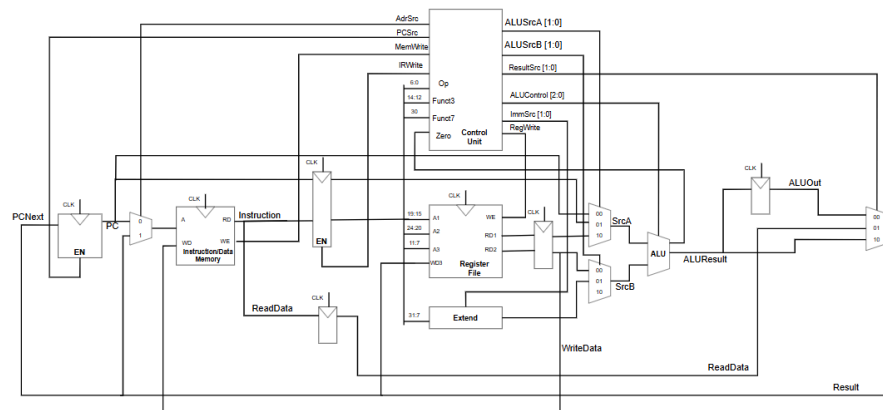
R-type:
Fetch → Decode → Execute → Write Back

Trong khi:

lw:
Fetch → Decode → Address → Memory → Write Back

Nhờ chia nhỏ quá trình thực thi, chu kỳ clock của Multicycle có thể ngắn hơn so với Single-Cycle.

3.2. Đặc điểm của kiến trúc Multicycle



Khác với Single-Cycle, trong Multicycle:

- Một instruction sử dụng nhiều chu kỳ clock.
- Các khối phần cứng có thể được tái sử dụng ở những chu kỳ khác nhau.
- Cần thêm các thanh ghi trung gian để lưu kết quả giữa các chu kỳ.
- Control Unit thường được xây dựng dưới dạng **Finite State Machine (FSM)**.
- Instruction Memory và Data Memory có thể được gộp thành một khối nhớ chung.

Ví dụ ALU có thể được dùng ở nhiều thời điểm:

Fetch:
ALU tính $PC + 4$

Decode:
ALU tính $PC + \text{immediate}$

Execute:
ALU thực hiện phép toán của instruction

3.3. Bộ nhớ dùng chung Instruction/Data Memory

Trong Single-Cycle, Instruction Memory và Data Memory thường được tách riêng vì có thể cần truy cập trong cùng một chu kỳ.

Trong Multicycle, việc lấy instruction và truy cập dữ liệu xảy ra ở các chu kỳ khác nhau. Vì vậy có thể sử dụng một khối memory chung.

Địa chỉ memory được chọn bởi tín hiệu **AdrSrc**:

```
AdrSrc = 0 → Address = PC
AdrSrc = 1 → Address = ALUOut
```

Trong trạng thái Fetch:

```
Address = PC
```

Còn đối với **Lw** hoặc **Sw**:

```
Address = ALUOut
```

Trong đó **ALUOut** giữ địa chỉ dữ liệu đã được ALU tính từ chu kỳ trước.

3.4. Các thanh ghi trung gian

Do một instruction trải qua nhiều chu kỳ nên các kết quả tạm phải được lưu lại.

Một số thanh ghi chính gồm:

Instruction Register

IR lưu instruction được đọc từ memory trong trạng thái Fetch.

```
IR ← Memory[PC]
```

OldPC

OldPC lưu địa chỉ của instruction hiện tại.

Trong Fetch:

```
OldPC ← PC
PC ← PC + 4
```

Sau Fetch:

```
OldPC = địa chỉ instruction đang thực hiện
PC = địa chỉ instruction tuần tự kế tiếp
```

A và B

Hai thanh ghi này lưu dữ liệu đọc từ Register File:

```
A ← RD1
B ← RD2
```

ALUOut

Lưu kết quả của ALU để sử dụng ở chu kỳ sau.

Ví dụ:

```
ALUOut ← rs1 + immediate
```

hoặc:

```
ALUOut ← rs1 + rs2
```

Data/MDR

Lưu dữ liệu đọc từ memory:

```
Data ← Memory[ALUOut]
```

sau đó được sử dụng trong Write Back của `lw`.

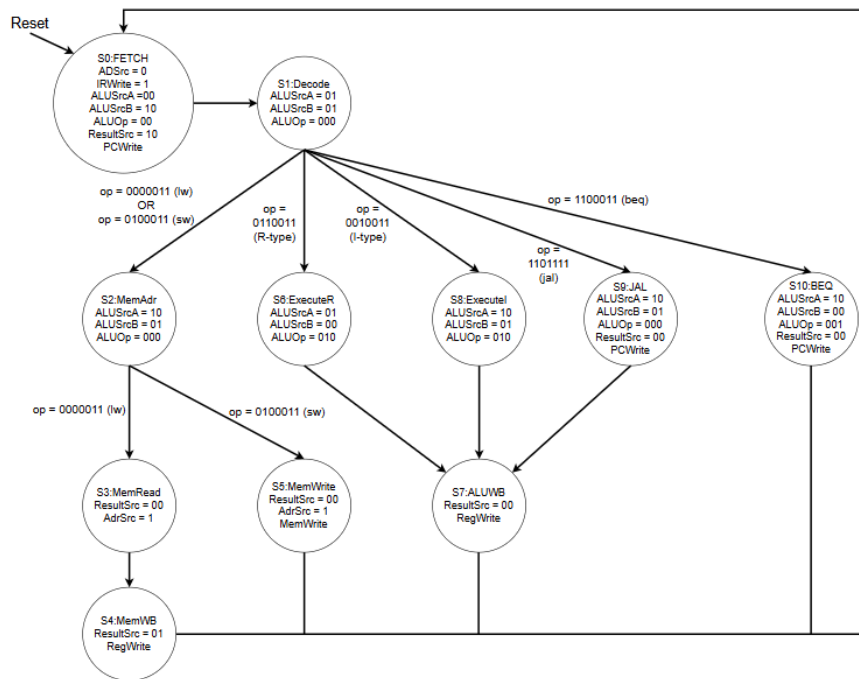
3.5. FSM điều khiển Multicycle

Control Unit của Multicycle thường được xây dựng bằng một FSM.

Một tập trạng thái cơ bản có thể gồm:

```
S0  FETCH
S1  DECODE
S2  MEM_ADR
S3  MEM_READ
S4  MEM_WB
S5  MEM_WRITE
S6  EXEC_R
S7  ALU_WB
S8  EXEC_I
S9  JAL
S10 BEQ
```

Mỗi trạng thái tạo ra một nhóm tín hiệu điều khiển khác nhau.



3.6. Hoạt động của các trạng thái chính

FETCH

Trong trạng thái Fetch:

```

IR ← Memory[PC]
OldPC ← PC
PC ← PC + 4

```

Các tín hiệu chính:

```

AdSrc = 0
IRWrite = 1
PCUpdate = 1
ALU thực hiện ADD

```

DECODE

Trong trạng thái Decode:

```

A ← Register[rs1]
B ← Register[rs2]

```

Đồng thời có thể tính trước địa chỉ đích:

```

ALUOut ← OldPC + ImmExt

```

Giá trị này được sử dụng cho `beq` hoặc `jal`.

MEM_ADR

Đối với `lw` và `sw`, ALU tính địa chỉ dữ liệu:

```
ALUOut ← A + ImmExt
```

Sau đó FSM chuyển sang trạng thái đọc hoặc ghi memory.

MEM_READ

Đối với `lw`:

```
Data ← Memory[ALUOut]
```

Sau đó chuyển tới `MEM_WB`.

MEM_WB

Dữ liệu đọc từ memory được ghi vào Register File:

```
Register[rd] ← Data
```

MEM_WRITE

Đối với `sw`:

```
Memory[ALUOut] ← B
```

Lệnh `sw` kết thúc tại đây và không cần Write Back.

EXEC_R

Đối với lệnh R-type:

```
ALUOut ← A op B
```

Ví dụ:

```
ADD  
SUB  
AND  
OR  
SLT
```

Sau đó chuyển tới `ALU_WB`.

EXEC_I

Đối với lệnh I-type:

```
ALUOut ← A op ImmExt
```

Ví dụ:

```
addi
```

ALU thực hiện:

```
A + ImmExt
```

ALU_WB

Kết quả ALU được ghi về Register File:

```
Register[rd] ← ALUOut
```

Trạng thái này được dùng cho R-type, I-type và có thể dùng cho phần write-back của jal.

BEQ

Đối với:

```
beq rs1, rs2, offset
```

ALU thực hiện:

```
A - B
```

Nếu:

```
Zero = 1
```

thì PC nhận địa chỉ đích đã được lưu trong ALUOut :

```
PC ← ALUOut
```

Có thể tạo điều kiện ghi PC:

```
PCWrite = Branch & Zero
```

JAL

Lệnh:

```
jal rd, offset
```

thực hiện:

```
PC ← PCTarget  
rd ← OldPC + 4
```

Trong Decode, địa chỉ đích đã được tính:

```
ALUOut ← OldPC + ImmExt
```

Trong trạng thái JAL:

```
PC ← ALUOut
```

đồng thời ALU tính:

```
OldPC + 4
```

để sau đó ghi vào `rd`.

3.7. Control Unit trong Multicycle

Control Unit có thể chia thành ba phần:

```
Main FSM  
Instruction Decoder  
ALU Decoder
```

FSM tạo các tín hiệu như:

```
PCUpdate  
Branch  
RegWrite  
MemWrite  
IRWrite  
AdrSrc  
ResultSrc  
ALUSrcA  
ALUSrcB  
ALUOp
```

Instruction Decoder tạo `ImmSrc`, còn ALU Decoder tạo `ALUControl`.

Tín hiệu ghi PC có thể được tạo từ:

```
PCWrite=PCUpdate | (Branch&Zero);
```

Trong đó:

```
PCUpdate = 1 → Fetch hoặc JAL  
Branch   = 1 → BEQ
```

3.8. Số chu kỳ của các instruction

Tùy FSM, số chu kỳ có thể được tổ chức như sau:

Instruction	Chu trình
R-type	Fetch → Decode → ExecR → ALUWB
I-type	Fetch → Decode → ExecI → ALUWB
lw	Fetch → Decode → MemAdr → MemRead → MemWB
sw	Fetch → Decode → MemAdr → MemWrite
beq	Fetch → Decode → BEQ
jal	Fetch → Decode → JAL → ALUWB

Do đó CPI của Multicycle không cố định bằng 1 mà phụ thuộc vào instruction.

3.9. Ưu và nhược điểm

Ưu điểm:

- Chu kỳ clock ngắn hơn Single-Cycle.
- Có thể tái sử dụng ALU và các khối phần cứng.
- Có thể dùng chung Instruction/Data Memory.
- Giảm lãng phí phần cứng.

Nhược điểm:

- Mỗi instruction cần nhiều cycle.
- CPI lớn hơn 1.
- Control Unit phức tạp hơn do phải dùng FSM.
- Cần nhiều thanh ghi trung gian để giữ trạng thái và dữ liệu.

Chương 4. Bộ xử lý RISC-V Pipeline

4.1. Khái niệm

Bộ xử lý **Pipeline** chia quá trình thực thi instruction thành nhiều giai đoạn và cho phép nhiều instruction được xử lý chồng lấp theo thời gian.


```
PCPlus4F = PCF + 4
```

PC tiếp theo được chọn giữa:

```
PC + 4
```

và địa chỉ branch/jump:

```
PCTargetE
```

Instruction Decode - ID

Ở Decode, instruction được giải mã và Register File được đọc.

Các trường chính:

```
rs1 = InstructionD[19:15]
rs2 = InstructionD[24:20]
rd  = InstructionD[11:7]
```

Control Unit tạo các tín hiệu:

```
RegWriteD
ResultSrcD
MemWriteD
JumpD
BranchD
ALUControlD
ALUSrcD
ImmSrcD
```

Khối Extend đồng thời tạo:

```
ImmExtD
```

Execute - EX

Ở Execute, ALU thực hiện phép toán của instruction.

Ví dụ:

```
R-type:
ALUResultE = SrcAE op SrcBE
```

với I-type:

```
ALUResultE = SrcAE op ImmExtE
```

Đối với branch, stage này cũng thực hiện so sánh và tính địa chỉ đích:

```
PCTargetE = PCE + ImmExtE
```

Tín hiệu chọn PC có thể được tạo bởi:

```
PCSrcE=JumpE| (BranchE&ZeroE);
```

Memory - MEM

Stage Memory được sử dụng chủ yếu cho `lw` và `sw`.

Với:

```
lw rd, imm(rs1)
```

thực hiện:

```
ReadDataM = DataMemory[ALUResultM]
```

Với:

```
sw rs2, imm(rs1)
```

thực hiện:

```
DataMemory[ALUResultM] = WriteDataM
```

Write Back - WB

Ở stage Write Back, dữ liệu được chọn để ghi về Register File.

MUX `ResultSrcW` chọn:

```
00 → ALUResultW  
01 → ReadDataW  
10 → PCPlus4W
```

Sau đó:

```
Register[RdW] ← ResultW
```

nếu:

```
RegWriteW = 1
```

4.3. Các Pipeline Register

Để giữ dữ liệu giữa các stage, cần các pipeline register:

```
F/D  
D/E  
E/M  
M/W
```

Register F/D

Lưu:

```
InstructionD
PCD
PCPlus4D
```

Register D/E

Lưu:

```
RD1E
RD2E
PCE
ImmExtE
PCPlus4E

Rs1E
Rs2E
RdE
```

cùng các control:

```
RegWriteE
ResultSrcE
MemWriteE
JumpE
BranchE
ALUControlE
ALUSrcE
```

Register E/M

Lưu:

```
ALUResultM
WriteDataM
PCPlus4M
RdM
```

và:

```
RegWriteM
ResultSrcM
MemWriteM
```

Register M/W

Lưu:

```
ALUResultW
ReadDataW
PCPlus4W
RdW
```

cùng:

```
RegWriteW
ResultSrcW
```

Một nguyên tắc quan trọng là:

| Control signal phải đi chuyển cùng instruction tương ứng qua pipeline.

4.4. Data Hazard

Do nhiều instruction được thực hiện chồng lấp, instruction sau có thể cần dữ liệu mà instruction trước chưa ghi về Register File.

Ví dụ:

```
add x5, x1, x2
sub x6, x5, x3
```

Lệnh `sub` cần `x5` ngay khi kết quả của `add` chưa đến Write Back.

Đây là **data hazard**.

4.5. Forwarding

Để giảm stall, kết quả có thể được chuyển trực tiếp từ các stage sau về Execute.

Các tín hiệu:

```
ForwardAE
ForwardBE
```

có thể được quy ước:

```
00 → dùng RD1E/RD2E
01 → dùng ResultW
10 → dùng ALUResultM
```

Điều kiện forwarding cho toán hạng A:

```
ForwardAE=
  (RegWriteM&& (RdM!=0)&& (Rs1E==RdM)) ? 2'b10 :
  (RegWriteW&& (RdW!=0)&& (Rs1E==RdW)) ? 2'b01 : 2'b00;
```

Tương tự cho `ForwardBE`.

Nếu cả Memory và Write Back cùng có thể cung cấp dữ liệu, kết quả từ Memory được ưu tiên vì mới hơn.

4.6. Load-Use Hazard

Forwarding không thể giải quyết hoàn toàn trường hợp:

```
lw x5, 0(x1)
add x6, x5, x2
```

Dữ liệu của `lw` chỉ có sau stage Memory, trong khi `add` cần nó tại Execute ngay cycle kế tiếp.

Điều kiện phát hiện có thể là:

```
lwStall=
  (RdE!=0)&&
```

```
((Rs1D==RdE) || (Rs2D==RdE)) && ResultSrcE[0];
```

Khi xảy ra load-use hazard:

```
StallF = 1  
StallD = 1  
FlushE = 1
```

4.7. Stall và Bubble

StallF làm PC giữ nguyên:

```
StallF = 1 → PC không cập nhật
```

StallD làm thanh ghi F/D giữ nguyên instruction:

```
StallD = 1 → Decode không nhận instruction mới
```

FlushE đưa một bubble vào Execute bằng cách xóa các control signal:

```
RegWriteE = 0  
MemWriteE = 0  
BranchE   = 0  
JumpE     = 0
```

Nhờ đó instruction phụ thuộc được trì hoãn một cycle cho đến khi dữ liệu sẵn sàng.

4.8. Control Hazard

Các lệnh thay đổi luồng chương trình như:

```
beq  
jal
```

tạo ra **control hazard**.

Trong khi branch đang được quyết định ở Execute, CPU đã fetch các instruction phía sau theo **PC + 4**.

Nếu branch hoặc jump được lấy:

```
PCSrcE = 1
```

các instruction đã đi vào pipeline theo đường sai phải bị xóa.

Logic:

```
FlushD=PCSrcE; FlushE=1<Stall|PCSrcE;
```

Trong đó:

```
FlushD = 1 → xóa instruction ở Decode  
FlushE = 1 → xóa instruction đi vào Execute
```

4.9. Hazard Unit

Hazard Unit kết hợp logic forwarding, stall và flush.

Các đầu vào chính:

```
Rs1E
Rs2E
RdE
RdM
RdW

RegWriteM
RegWriteW
ResultSrcE
InstructionD
PCSrcE
```

Các đầu ra:

```
ForwardAE
ForwardBE
StallF
StallD
FlushD
FlushE
```

Do đó Hazard Unit là thành phần quan trọng giúp pipeline duy trì kết quả đúng khi nhiều instruction phụ thuộc lẫn nhau.

4.10. Hiệu năng của Pipeline

Với pipeline 5 stage, một instruction vẫn cần nhiều stage để hoàn thành, nhưng nhiều instruction được thực hiện song song.

Ở trạng thái lý tưởng: $CPI=1$. Sau khi pipeline được lấp đầy.

Tuy nhiên CPI thực tế có thể tăng do:

```
stall
branch
jump
flush
```

Thời gian thực thi có thể biểu diễn gần đúng: $CPU\ Time = Instruction\ Count \times CPI \times Clock\ Period$

Ưu điểm chính của pipeline không phải giảm latency của một instruction riêng lẻ mà là tăng **throughput**, tức số instruction hoàn thành trong một khoảng thời gian.

4.11. Ưu và nhược điểm

Ưu điểm:

- Cho phép nhiều instruction được xử lý đồng thời.
- Tăng throughput đáng kể.
- Chu kỳ clock ngắn hơn Single-Cycle.
- Sau khi pipeline đầy, CPI có thể tiến gần 1.

Nhược điểm:

- Datapath và Control Unit phức tạp hơn.
- Phải xử lý data hazard và control hazard.
- Cần forwarding, stall và flush.
- Cần thêm nhiều pipeline register.

Chương 5. Hazard trong Pipeline

5.1. Khái niệm Hazard

Trong kiến trúc Pipeline, nhiều instruction được xử lý đồng thời ở các stage khác nhau. Điều này giúp tăng throughput nhưng cũng tạo ra các tình huống mà một instruction không thể thực hiện đúng ở chu kỳ kế tiếp.

Các tình huống này được gọi là **hazard**.

Ba loại hazard phổ biến gồm:

```
Structural Hazard
Data Hazard
Control Hazard
```

Trong thiết kế pipeline này, phần xử lý chính tập trung vào:

- Data Hazard
- Load-Use Hazard
- Control Hazard

5.2. Data Hazard

Data Hazard xảy ra khi instruction phía sau cần sử dụng kết quả của instruction phía trước nhưng kết quả đó chưa được ghi về Register File.

Ví dụ:

```
add x5, x1, x2
sub x6, x5, x3
```

Lệnh `sub` cần giá trị mới của `x5`, nhưng tại thời điểm `sub` vào stage Execute, lệnh `add` có thể vẫn chưa đến Write Back.

Nếu không xử lý hazard, `sub` có thể đọc giá trị cũ của `x5`, dẫn đến kết quả sai.

5.3. Forwarding

Để giải quyết phần lớn Data Hazard, pipeline sử dụng cơ chế **Forwarding**.

Thay vì chờ kết quả được ghi về Register File, dữ liệu được chuyển trực tiếp từ stage phía sau về đầu vào ALU tại Execute.

Hai tín hiệu điều khiển forwarding là:

```
ForwardAE
ForwardBE
```

Quy ước:

```
00 → dùng RD1E hoặc RD2E
01 → dùng ResultW
10 → dùng ALUResultM
```

Logic cho toán hạng A:

```
assignForwardAE=
  (RegWriteM&& (RdM!=5'd0)&& (Rs1E==RdM)) ? 2'b10 :
  (RegWriteW&& (RdW!=5'd0)&& (Rs1E==RdW)) ? 2'b01 : 2'b00;
```

Tương tự với toán hạng B:

```
assignForwardBE=
  (RegWriteM&& (RdM!=5'd0)&& (Rs2E==RdM)) ? 2'b10 :
  (RegWriteW&& (RdW!=5'd0)&& (Rs2E==RdW)) ? 2'b01 : 2'b00;
```

Nếu cả stage M và W đều có thể cung cấp cùng một thanh ghi, dữ liệu từ stage M được ưu tiên vì đây là giá trị mới hơn.

5.4. Load-Use Hazard

Forwarding không giải quyết được hoàn toàn trường hợp một instruction sử dụng ngay dữ liệu vừa được load từ memory.

Ví dụ:

```
lw x5, 0(x1)
add x6, x5, x2
```

Lệnh `lw` chỉ nhận được dữ liệu sau stage Memory, trong khi `add` cần `x5` tại stage Execute ngay ở chu kỳ kế tiếp.

Do đó cần phát hiện Load-Use Hazard.

Điều kiện:

```
assignlwStall=
  (RdE!=5'd0)&&
  ((Rs1D==RdE) || (Rs2D==RdE))&&ResultSrcE[0];
```

Trong đó:

RdE

là thanh ghi đích của instruction đang ở Execute.

Rs1D, Rs2D

là các thanh ghi nguồn của instruction đang ở Decode.

Với quy ước:

ResultSrc = 01 → dữ liệu từ Data Memory

thì:

ResultSrcE[0] = 1

cho biết instruction ở Execute là `lw`.

5.5. Stall

Khi xảy ra Load-Use Hazard, pipeline phải tạm thời giữ instruction lại.

Hazard Unit tạo:

```
StallF = 1
StallD = 1
```

`StallF` làm PC giữ nguyên:

```
StallF = 1
→ PC không cập nhật
```

StallD làm pipeline register F/D giữ nguyên:

```
StallD = 1
→ instruction ở Decode không thay đổi
```

Như vậy instruction phụ thuộc sẽ chờ thêm một chu kỳ.

5.6. Bubble và FlushE

Ngoài việc giữ Fetch và Decode, stage Execute cần được đưa vào trạng thái rỗng để tránh instruction sai tiếp tục đi qua pipeline.

Điều này được thực hiện bằng:

```
FlushE = 1
```

Khi đó register D/E được xóa các control signal quan trọng:

```
RegWriteE = 0
MemWriteE = 0
BranchE    = 0
JumpE      = 0
```

Instruction bị thay bằng một **bubble**, nghĩa là nó không tạo ra thay đổi trạng thái kiến trúc.

Đối với Load-Use Hazard:

```
StallF = 1
StallD = 1
FlushE = 1
```

5.7. Control Hazard

Control Hazard xuất hiện với các instruction làm thay đổi PC, chẳng hạn:

```
beq
jal
```

Trong pipeline, branch hoặc jump thường được quyết định tại stage Execute. Tuy nhiên trước thời điểm đó, CPU đã fetch các instruction tiếp theo theo **PC + 4**.

Nếu branch hoặc jump được lấy, các instruction này nằm trên đường thực thi sai và phải bị xóa.

Tín hiệu:

```
PCSrcE = 1
```

cho biết PC phải chuyển sang địa chỉ đích.

Khi đó:

```
assignFlushD=PCSrcE; assignFlushE=!wStall|PCSrcE;
```

5.8. FlushD và FlushE

FlushD được dùng để xóa instruction đang ở Decode:

```
FlushD = 1  
→ xóa Register F/D
```

FlushE được dùng để xóa instruction đi vào Execute:

```
FlushE = 1  
→ xóa Register D/E
```

Khi branch hoặc jump được lấy:

```
PCSrcE = 1
```

thì:

```
FlushD = 1  
FlushE = 1
```

Hai instruction phía sau bị loại bỏ khỏi pipeline.

5.9. Hazard Unit

Hazard Unit là khối tổng hợp toàn bộ logic xử lý hazard.

Các đầu vào chính gồm:

```
Rs1E  
Rs2E  
RdE  
RdM  
RdW  
RegWriteM  
RegWriteW  
ResultSrcE  
InstructionD  
PCSrcE
```

Các đầu ra:

```
ForwardAE  
ForwardBE  
StallF  
StallD  
FlushD  
FlushE
```

Có thể biểu diễn chức năng:

5.10. Ví dụ xử lý Forwarding

Xét:

```
add x5, x1, x2  
sub x6, x5, x3
```

Khi `sub` ở Execute và `add` ở Memory:

```
Rs1E = x5  
RdM = x5  
RegWriteM = 1
```

Hazard Unit tạo:

```
ForwardAE = 10
```

MUX đầu vào ALU chọn:

```
ALUResultM
```

thay vì giá trị cũ từ Register File.

Do đó không cần stall.

5.11. Ví dụ xử lý Load-Use

Xét:

```
lw x5, 0(x1)  
add x6, x5, x2
```

Khi `Lw` ở Execute và `add` ở Decode:

```
RdE = x5  
Rs1D = x5  
ResultSrcE = 01
```

nên:

```
lwStall = 1
```

Hazard Unit tạo:

```
StallF = 1  
StallD = 1  
FlushE = 1
```

Sau một chu kỳ chờ, dữ liệu load đã sẵn sàng và có thể được forward từ Write Back về Execute.

5.12. Ví dụ xử lý Branch

Xét:

```
beq x1, x2, target
```

Nếu tại Execute:

```
x1 = x2
```

thì ALU tạo:

```
ZeroE = 1
```

và:

```
PCSrcE=JumpE| (BranchE&ZeroE);
```

suy ra:

```
PCSrcE = 1
```

Hazard Unit tạo:

```
FlushD = 1  
FlushE = 1
```

để loại bỏ các instruction đã được fetch theo đường sai.

5.13. Ảnh hưởng của Hazard đến hiệu năng

Pipeline lý tưởng có thể đạt:

$CPI \approx 1$

sau khi được lấp đầy.

Tuy nhiên hazard có thể làm CPI tăng.

Forwarding giúp giảm số cycle phải chờ, nhưng một số trường hợp như Load-Use vẫn cần stall.

Branch và jump cũng gây mất các cycle do các instruction sai đường phải bị flush.

Do đó hiệu năng pipeline thực tế phụ thuộc nhiều vào:

- số lượng data dependency,
- số lệnh load liên tiếp,
- tần suất branch và jump,
- hiệu quả của forwarding và hazard detection.